

# SkillRouter: 面向大规模 LLM 智能体的技能路由

YanZhao Zheng   ZhenTao Zhang   Chao Ma   YuanQiang Yu   JiHuai Zhu   Wu Yong  
Tianze Xu   Baohua Dong\*   Hangcheng Zhu   Ruohui Huang   Gang Yu

Alibaba Group, Hangzhou, China (阿里巴巴集团, 中国杭州)

预印本, 审稿中 (Preprint. Under review.)   arXiv:2603.22455v4 [cs.LG] 1 Apr 2026

邮箱: {zhengyanzhao.zyz, zhangzhentao.zzt, mc524716, yuyuanqiang.yyq, zhujihuai.zjh, wy517954, xutianze.xtz, baohua.dbh, linran.lr09, wentong, ruohai}@alibaba-inc.com

\* 通讯作者。 代码: <https://github.com/zhengyanzhao1997/SkillRouter>

## 摘要

可复用技能 (reusable skill) 使 LLM 智能体能够将任务专用的流程、工具可供性 (tool affordance) 以及执行指引打包为模块化的构建单元。随着技能生态规模增长到数万个条目, 在推理时把每个技能都暴露给模型变得不可行。这就产生了一个技能路由 (skill-routing) 问题: 给定一个用户任务, 系统必须在下流的规划或执行之前识别出相关的技能。现有的智能体技术栈通常依赖渐进式披露 (progressive disclosure), 即只暴露技能的名称和描述, 而隐藏完整的实现主体 (implementation body)。我们在一个由 SkillsBench 派生、包含约 80K 个候选技能的基准上考察这一设计选择, 针对的是一个实践上重要的场景: 大规模、高度重叠的技能注册库 (skill registry)。在该场景下对具有代表性的稀疏检索、稠密检索与重排 (reranking) 基线进行评估时, 隐藏技能主体会导致路由准确率下降 31–44 个百分点, 这表明在该场景下完整的技能文本是一个关键的路由信号, 而非仅仅是元数据上的细微补充。受此发现的启发, 我们提出了 SkillRouter, 一个紧凑的、参数量为 1.2B 的全文本「检索-重排」(retrieve-and-rerank) 流水线。SkillRouter 在我们的基准上达到 74.0% 的 Hit@1——在我们所评估的基线中取得了最强的平均 top-1 路由性能——同时使用的参数量减少为原来的 1/13, 运行速度比最强的基础流水线快 5.8 倍。其排序增益进一步推广到了一个由三个技能来源独立构建的补充基准上。在一项跨四个编程智能体 (coding agent) 的互补式端到端研究中, 路由增益转化为任务成功率的提升, 且对能力更强的智能体增益更大。

## 1 引言

技能 (skill) 已成为一种实用的抽象, 用于以可复用的流程、工具知识与执行指引来扩展 LLM 智能体。近期的编程智能体产品, 如 Claude Code、Codex 和 OpenClaw, 都将可复用技能作为一等能力来暴露 (Anthropic, 2025; OpenAI, 2025; OpenClaw, 2026b)。这些系统反映了技能注册库在真实部署中日益增长的使用。把每个技能都呈现给智能体是不可行的, 因此真实系统需要技能路由: 给定一

个用户任务，从一个大规模技能池中检索出正确的技能。这一场景具有一个重要的不对称性：路由组件可以查看完整的技能文本，而最终消费该技能的智能体通常只能看到其名称和描述。在已部署的智能体技术栈中，这个上游的路由决策是一个高杠杆的瓶颈：一旦浮现出了错误的技能候选名单，下游的规划与执行就几乎没有机会恢复。因此，问题不仅在于智能体能否使用一个被提供的技能，更在于系统能否在严重的池规模混淆（pool-scale confusion）下找到正确的技能。

当前的智能体框架隐含地把元数据当作足以用于选择，然而这一假设尚未在真实规模下得到检验。现有的基准，如 SkillsBench (Li et al., 2026b)、ToolBench (Qin et al., 2023) 和 MetaTool (Huang et al., 2024)，研究的是下游的工具使用或工具选择行为，但它们并未直接评估隐藏实现下的大池上游技能路由。在检索一侧，已有工作研究了重排和上下文感知检索 (Zheng et al., 2024; Yuan et al., 2024)，但通常只在名称-描述元数据上进行，且候选池要小得多。这就在当前的基准实践与现实的智能体部署之间留下了一道鸿沟——在现实部署中，技能注册库可能既庞大又高度重叠。我们的目标并不是声称每一个技能路由基准都表现出相同的失败模式，而是研究一个实践上重要的场景：大规模、高度重叠的技能注册库，在这种场景下许多候选都可能对同一个查询显得相关。

我们在一个包含约 80K 个技能、以及 75 条经专家验证、由 SkillsBench 派生的查询的基准上研究技能路由，这些查询实例化了上述场景。我们核心的经验性发现是：在该场景下，完整的技能文本是一个关键的路由信号——移除技能主体会使具有代表性的稀疏、稠密与重排基线下降 31–44pp（百分点），而长度受控的注意力诊断与描述质量分层分析则反对了简单的「仅长度」或「描述质量」解释。受此观察启发，我们构建了 SkillRouter，一个紧凑的、1.2B 参数的全文本「检索-重排」流水线。主配置（1.2B：0.6B 编码器 + 0.6B 重排器）达到 74.0% 的 Hit@1 与 70.4% 的 R@10，相比之下最强的 16B 基础流水线为 68.0% 的 Hit@1——在参数量减少为原来的 1/13、服务延迟降低 5.8 倍的情况下取得了相当或更高的准确率。一个 8B 的放大版本达到 76.0%。我们还验证了超越检索指标之外的迁移性：在一项使用自然池、跨四个编程智能体的互补式端到端研究中，SkillRouter 在 top-1 和 top-10 两种设置下都相对最强基础路由器提升了平均任务成功率，且对能力更强的智能体增益更明显。这些下游结果应被解读为端到端的效用度量，而非对穷尽式黄金技能（gold-skill）恢复的直接代理，因为智能体消费的是一个有界的候选名单，而不是抽象标注的集合。在一项真实池的 GPU 基准上，1.2B 流水线以亚秒级的中位延迟服务查询。我们还在 SkillBench-Supp 上评估了相同的检查点（checkpoint），这是一个由三个技能来源构建、包含 256 条查询的独立基准，结果表明所观察到的增益并不特定于那 75 条查询的核心基准。

我们的贡献有三方面：

1. 在两个互补的、覆盖约 80K 技能池的基准上——一个是包含 75 条查询、带有 Easy/Hard 鲁棒性分层的专家验证核心集，另一个是由三个技能来源独立构建、包含 256 条查询的补充集——我们表明完整的技能文本是一个关键的路由信号（第 3 节），并表明长度受控的诊断与描述质量分层分析反对了简单的「仅长度」或「描述质量」解释。
2. 我们提出 SkillRouter，一个由标准信息检索 (IR) 组件构建的紧凑全文本「检索-重排」流水线，并识别出在同质技能池中特别必要的两项训练适配：用于处理近重复技能的假阴性过滤 (false-negative filtering)，以及用于解决细粒度候选竞争的列表式 (listwise) 重排损失。

3. 我们表明路由增益迁移到了一项使用自然池、跨四个编程智能体的互补式端到端研究中，并刻画了紧凑流水线在真实池 GPU 服务基准上的效率-准确率权衡。

## 2 问题定义与基准

**任务与指标。**我们研究技能路由：给定一个任务查询  $q$  和一个大规模技能池  $S = \{s_1, \dots, s_N\}$ ，检索出解决该任务所需的技能集合  $G_q \subseteq S$ 。每个技能包含一个名称、一段描述和一个完整的实现主体。这就产生了一个隐藏主体 (hidden-body) 的不对称性：路由系统可以查看完整的技能文本，而下游智能体通常只能看到元数据。我们报告 Hit@1 作为主要的 top-1 路由指标，同时报告 MRR@10、Recall@K ( $K \in \{10, 20, 50\}$ ；恢复出的黄金技能的平均比例) 以及 FC@10 (其完整黄金技能集出现在前 10 名中的查询所占比例)。对于多技能查询，Hit@1 机械地定义为是否有任意一个所需技能被排在第一位。因此我们额外报告 Recall@K 和 FC@10，以更直接地刻画候选名单与完整集合的覆盖率。

**表 1:** 示意性的基准样例。困难干扰项在主题上仍然看似合理，但无法满足所需的功能。

| 类型                      | 名称                  | 描述                                |
|-------------------------|---------------------|-----------------------------------|
| 黄金标准 (Ground truth)     | speech-to-text      | 使用 Whisper 在本地转录音频/视频，并返回带时间戳的文本。 |
| 池内干扰项 (Pool distractor) | audio-transcriber   | 面向已上传音频文件的通用云端转录服务。               |
| 困难干扰项 (Hard distractor) | video-subtitle-sync | 利用音频线索将字幕时间同步到视频播放。               |

**基准构建。**我们基于 SkillsBench (Li et al., 2026b) 构建该基准，它提供了专家精选的「任务-技能」映射。从 87 个 SkillsBench 任务出发，我们排除了 12 个仅含通用技能的样例（其标签只包含文件类型技能，如 pdf 或 xlsx），保留 75 条核心查询：24 条单技能查询和 51 条多技能查询。我们针对一个约 80K 技能的池进行评估，该池由 SkillsBench 技能加上一个大规模开源技能集合组装而成，后者涵盖 51 个类别，取自 Claude Skill Registry Core (Majiayu000, 2026)。为了探究鲁棒性，我们报告两个分层：Easy (78,361 个候选技能) 和 Hard (在加入 780 个由 LLM 生成、主题相关但功能不同的干扰技能后，共 79,141 个候选)。所有主结果均对 Easy 和 Hard 取平均；附录 A 报告了精确的核心查询选择协议以及对 80K 池的元数据审计，而附录 B 详述了干扰项的生成与代表性数据样例。

**基准可信度与范围。**SkillsBench 提供的是专家精选的「任务-技能」映射，而非弱推断的标签。Easy/Hard 划分离开了两种失败模式：Easy 中是标准的大池检索，Hard 中是在功能接近但不正确的备选项之间的混淆。表 1 说明了这一设计：困难干扰项是同领域、同技术或被过度泛化的备选项，它们表面上仍然看似合理，但无法满足所需功能，并作为针对功能级混淆的定向压力测试，而非对自然仓库中干扰项普遍程度的估计。这 75 条核心查询跨越 8 个超级类别下的 55 个应用领域，且没有任何单一超级类别超过 17% (附录 A)。这针对的是一个实践上重要的场景：大规模、高度重叠的技能注

册库，这在社区生态与内部工具目录中很常见。

### 3 哪些信号驱动了技能选择？

当前的智能体框架通常只暴露技能的名称和描述，隐含地假设元数据足以用于选择。我们在本文的主基准设置上检验这一假设，报告正文其余部分所使用的 Easy/Hard 平均值。图 1（左）使用了与主表对齐的三个代表性基线：BM25、最强的纯编码器基础模型（Qwen3-Emb-8B），以及最强的基础「检索-重排」流水线（Qwen3-Emb-8B × Qwen3-Rank-8B）。附录表 9 报告了在相同的 75 条核心查询上，BM25、Qwen3-Emb-0.6B 和 Qwen3-Emb-8B 之间纯编码器的完整比较，即仅元数据输入（仅名称 + 描述；「nd」）与全文本输入的对比。我们还分析了交叉编码器（cross-encoder）的注意力，并对字段长度加以控制，以检验重排器在最终决策中是否只是单纯地跟随字段长度。

**移除技能主体使各方法族的性能崩塌。**图 1（左）报告了三个代表性基线 31.4–44.0pp 的 Hit@1 下降。在本文的主 Easy/Hard 平均上，BM25 从 31.4% 跌至 0.0%，Qwen3-Emb-8B 从 64.0% 跌至 25.3%，Qwen3-Emb-8B × Qwen3-Rank-8B 从 68.0% 跌至 24.0%。附录表 9 显示 Qwen3-Emb-0.6B 也呈现相同的纯编码器模式，在同一基准上从 56.0% 跌至 18.7%。因此这种崩塌并不与单一的模型选择绑定：跨越稀疏检索、纯编码器检索与重排，移除技能主体都移除了一个关键的路由信号，并急剧降低了顶端排序性能。

**长度受控的注意力支持同一结论。**原始注意力质量受到长度的混淆，因为在所分析的 75 个「查询-技能」对中，主体、名称和描述字段分别占技能 token 的 96.5%、3.0% 和 0.5%。因此我们不会孤立地解读 91.7% 的总体主体注意力。相反，有信息量的信号是注意力在各字段上随层数的重新分配。如果重排器主要是对字段长度作出响应，那么注意力在整个网络中应保持接近 token 占比基线。事实并非如此：名称字段仅占技能 token 的 3.0%，却在第 19 层升至 26.3% 的注意力，随后最终层回落到 98.1% 的主体注意力。最终层的主体注意力在 69/75 条查询上超过了主体的 token 占比，且与绝对主体长度几乎不相关（ $r = 0.04$ ）。综合来看，这些诊断使简单的「仅长度」解释变得不太可能，并支持以下结论：技能主体贡献了超越原始字段长度的实质性路由信号。附录 C 给出了精确的计算方式，以及支撑这一「主体 → 名称 → 主体」轨迹的完整逐层与查询级诊断。作为进一步的对照，附录 D 按黄金标准（GT）描述长度对 nd→full 的差距进行分层，发现即使在描述最长的四分位（quartile）技能上，该差距仍然很大（≥26pp），从而反对了描述质量混淆因素的解释。

Preprint. Under review.

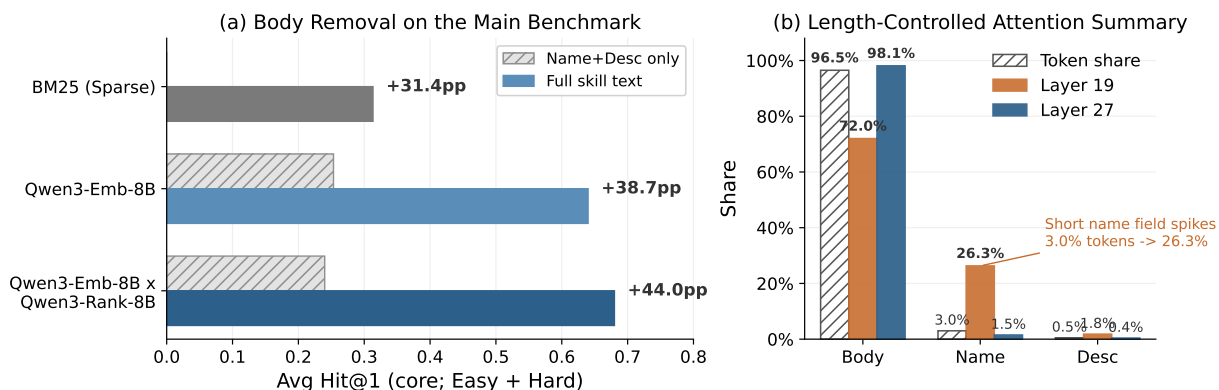


Figure 1: Full skill text is a critical routing signal. Left: Averaged over the paper’s Easy

图 1: 完整的技能文本是一个关键的路由信号。左: 在本文的 Easy 和 Hard 分层上取平均后, 移除主体使 BM25 的 Hit@1 下降 31.4pp, Qwen3-Emb-8B 下降 38.7pp, Qwen3-Emb-8B × Qwen3-Rank-8B 下降 44.0pp。右: 长度受控的注意力诊断反对了简单的「仅长度」解释: 尽管主体字段占技能 token 的 96.5%, 短小的名称字段在第 19 层却峰值达到 26.3% 的注意力 (尽管仅占 token 的 3.0%), 而最终层回落到 98.1% 的主体注意力。

**蕴含。**综合来看, 这些结果表明, 在该场景下, 无论是检索还是重排, 完整的技能文本都是实现可靠路由的关键信号。这一观察直接促成了下一节中 SkillRouter 的设计。

## 4 SkillRouter: 一个紧凑的全文本路由配方

受第 3 节中主体访问发现的启发, 我们提出 SkillRouter, 一个为大规模、同质技能池量身定制的紧凑全文本「检索-重排」流水线。它的主要贡献是一个针对特定场景的路由配方, 以及两项在该机制下能实质性提升性能的训练适配: 用于处理破坏对比学习的近重复技能的假阴性过滤, 以及用于解决主题相似候选之间细粒度竞争的列表式重排。我们并未引入新的编码器或重排器架构; 相反, 我们表明这些选择在该场景下很重要, 且由此得到的紧凑流水线占据了一个有利的效率-准确率前沿。

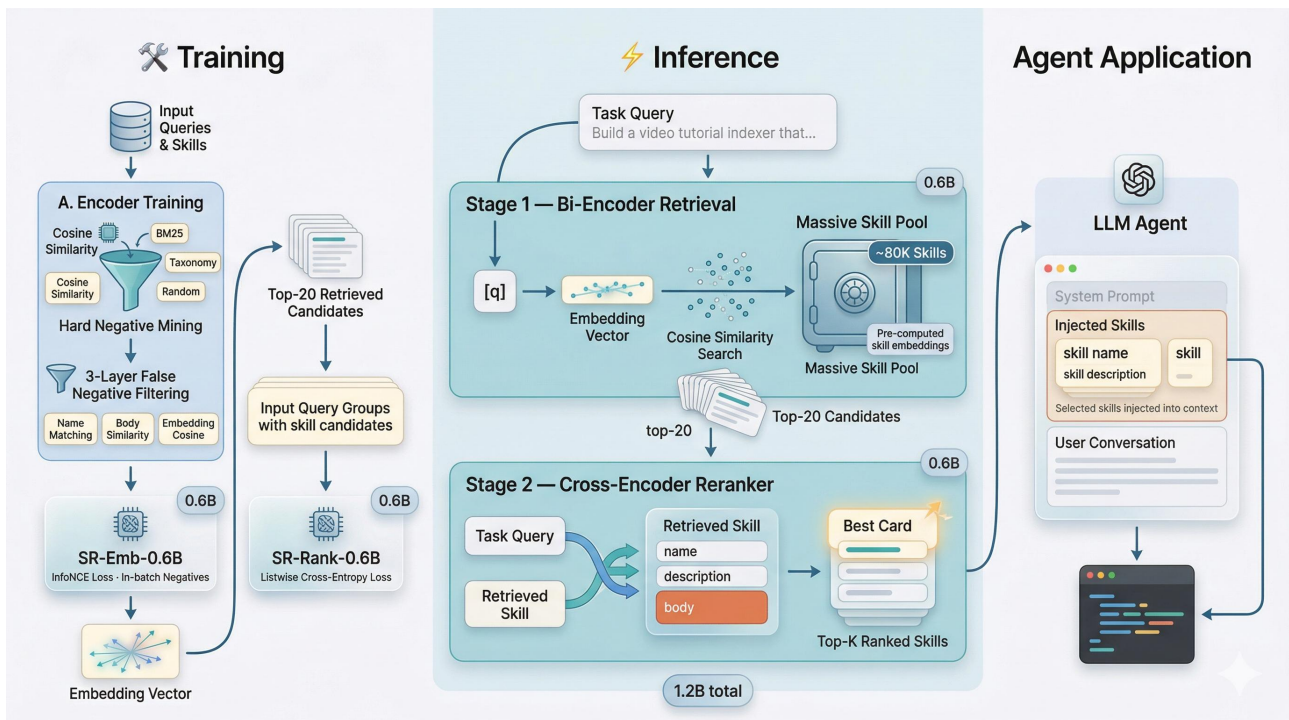


图 2: SkillRouter 流水线。一个双编码器（bi-encoder）从完整的约 80K 池中检索出 top-20 候选；一个交叉编码器对它们进行重排。两个阶段都使用完整的技能文本，其动机来自第 3 节中的主体访问发现。

具体而言，SkillRouter 是一个全文本两阶段流水线：双编码器首先从约 80K 池中检索出一个短候选名单，然后交叉编码器使用完整的技能主体对这些候选进行重排。我们的主配置使用一个 0.6B 编码器和一个 0.6B 重排器，参数总量为 1.2B。图 2 总结了训练设置与两阶段推理路径。

**双编码器检索。**我们在 37,979 个合成的（查询，技能）对上微调 Qwen3-Emb-0.6B (Zhang et al., 2025)。技能从约 80K 社区池中采样，采用分层采样 (stratified sampling) 以确保类别多样性。对于每个采样得到的技能，我们使用一个 LLM (GPT-4o-mini) 生成一个合成的用户查询，提示词中给出该技能的元数据和主体内容 (附录 E; 附录表 15)。提示词指示模型在不透露技能名称的情况下生成一个真实的任务描述，使得生成的查询反映的是功能需求而非词面身份。带有基准标签的技能被排除在训练监督之外，确保编码器学习的是可迁移的路由模式，而非记忆基准技能。我们用 in-batch InfoNCE 在完整技能文本上优化检索器。在推理时，编码器离线嵌入整个技能库，并仅检索 top-20 候选，从而为第二阶段提供一个范围较窄但仍具多样性的决策集。

**困难负例挖掘。**实践中，单个用户请求可能匹配数十个表面相关的技能——例如多个「git」或「docker」管理工具——而其中只有一个提供所需的特定能力。随机负例无法教会编码器做出这些细粒度区分。每个查询配以来自四个互补来源的 10 个负例：由基础编码器的嵌入检索出的语义近邻（每个查询 4 个）、经 BM25 评分得到的词法匹配（3 个）、来自同一技能类别的分类法干扰项（2 个），以及来自不同类别的随机负例（1 个）。这一混合迫使编码器同时区分语义相近的备选项、词法混淆项和同类别干扰项——而这正是完整技能主体访问在操作上变得至关重要之处。附录 E 提供了完整的挖掘流程。

**假阴性过滤。**由于上述困难负例是从一个「同一能力常被不同作者以不同名称独立实现」的池中挖掘出来的，被挖掘出的候选集中不可避免地包含与黄金标准功能等价的技能。将这些当作负例会破坏对比信号。我们应用一个三层过滤器：名称去重、主体文本重叠 (trigram Jaccard  $> 0.6$ ) 和嵌入相似度 ( $> 0.92$ )，移除约 10% 的被挖掘负例。第 5.3 节表明该过滤贡献了 +4.0pp 的 Hit@1。

**交叉编码器重排。**检索器将 top-20 候选提供给一个经微调的 Qwen3-Rank-0.6B (Zhang et al., 2025)，它使用完整的扁平化技能文本对每个「查询-技能」对评分。训练使用由 SR-Emb-0.6B 检索出的 32,283 个候选名单，每个名单包含 20 个带二元相关性标签的技能；并应用与编码器阶段相同的假阴性过滤流程。我们采用列表式交叉熵而非逐点 (pointwise) 二元分类：一旦检索器把池缩小到 20 个候选，剩余技能往往在主题上都看似合理，因此重排器必须将候选彼此对比，而不是独立地为每个候选评分。第 5.3 节表明列表式训练是必不可少的，比逐点变体高出 30.7pp 的 Hit@1。

**实现细节。**两个模型都在单块 GPU 上训练。在推理时，技能被离线预先嵌入；一次实时查询需要一次编码器前向传递、一次近似最近邻 (approximate nearest-neighbor) 搜索，以及对 20 个候选的重排。编码器负责大池召回，而重排器把其全文本能力用于相似候选之间的细粒度区分。训练细节和 top-K 消融实验见附录 F 和 G。

## 5 实验

### 5.1 设置

我们的主评估使用第 2 节中的基准：覆盖约 80K 技能池的 75 条核心查询，在 Easy 和 Hard 两个分层上评估，除非另有说明否则取平均。为评估超越该核心基准的泛化性，我们在第 5.4 节报告了一个补充基准上的结果。我们的主要指标是 Hit@1，以 MRR@10 作为次要排序指标。对于多技能查询，我们额外报告 Recall@K 和 FC@10 作为覆盖率指标：R@10 作为主要的候选名单覆盖率指标，而 FC@10 提供更严格的完整覆盖视角。主结果中所有模型都使用完整技能文本；nd 与 full 的对比在第 3 节中已总结。除非另有说明，重排器都在编码器的 top-20 候选名单上运作。

**输入格式与基线。**每个技能包含名称、描述和主体；我们使用 full (全部三个字段，按各模型的输入上限截断) 和 nd (仅名称 + 描述)。所有经调优的模型都使用 full 输入。

**编码器基线。**我们比较四个编码器族：

- 稀疏检索：在完整技能文本上的 BM25 (Robertson & Zaragoza, 2009)。
- 传统开源双编码器：E5-Large-v2 (Wang et al., 2022)、GTE-Large-v1.5 (Li et al., 2023) 和 BGE-Large-v1.5 (Xiao et al., 2024)。
- 基于解码器的编码器：Qwen3-Emb-0.6B、Qwen3-Emb-8B (Zhang et al., 2025) 和 NV-Embed-v2 (Lee et al., 2024)。
- 专有 API：OpenAI text-embedding-3-large (OpenAI, 2024b) 和 Gemini gemini-embedding-001 (Google, 2025)。



表 2 报告了每个族中的代表性模型；完整的 11 个模型网格见附录 I。

**重排器基线与我们的系统。**在重排方面，我们评估 Qwen3 基础重排器 (Zhang et al., 2025) 和列表式 LLM-as-judge (以 LLM 作为评判者) 基线，它们全都在编码器的 top-20 候选名单上运作。我们自己的系统包括作为主紧凑流水线的 SR-Emb-0.6B / SR-Rank-0.6B，外加用于检验配方迁移性的 8B 放大变体。该基准通过规模、重叠与词法不匹配对两个阶段都施加压力：编码器必须在类别重叠和众多看似合理的备选项中检索，而重排器必须在 top-20 窗口内对高度相似的候选进行排序。

**表 2:** 在 80K 技能路由基准上的纯编码器检索结果。经调优的 0.6B 编码器 (高亮) 超越了大 13 倍的基础编码器，表明在该场景下任务专用训练可补偿规模。E/H/A 分别表示 Easy/Hard/平均。R@20 反映了用于下游重排的候选覆盖率。

| 模型                     | 参数量         | E-Hit@1     | H-Hit@1     | A-Hit@1     | A-MRR@10    | A-R@20      |
|------------------------|-------------|-------------|-------------|-------------|-------------|-------------|
| BM25                   | –           | .347        | .280        | .314        | .365        | .365        |
| BGE-Large-v1.5         | 335M        | .613        | .587        | .600        | .653        | .668        |
| gemini-embedding-001   | –           | .613        | .560        | .587        | .650        | .687        |
| text-embedding-3-large | –           | .640        | .600        | .620        | .658        | .664        |
| Qwen3-Emb-0.6B         | 0.6B        | .587        | .533        | .560        | .638        | .637        |
| Qwen3-Emb-8B           | 8B          | .653        | .627        | .640        | .698        | .726        |
| <b>SR-Emb-0.6B</b>     | <b>0.6B</b> | <b>.667</b> | <b>.640</b> | <b>.654</b> | <b>.723</b> | <b>.754</b> |
| SR-Emb-8B              | 8B          | .693        | .667        | .680        | .731        | .777        |

**表 3:** 端到端「检索-重排」结果 (top-20 候选)。紧凑的 1.2B 经调优流水线 (高亮) 在非放大配置中达到最高的 Hit@1，以 1/13 的参数量超越了 16B 基础流水线。E/H/A 分别表示 Easy/Hard/平均。

| 编码器                | 重排器                 | E-Hit@1     | H-Hit@1     | A-Hit@1     | A-MRR@10    | A-R@10      |
|--------------------|---------------------|-------------|-------------|-------------|-------------|-------------|
| Qwen3-Emb-0.6B     | Qwen3-Rank-0.6B     | .653        | .627        | .640        | .684        | .604        |
| Qwen3-Emb-8B       | Qwen3-Rank-0.6B     | .613        | .547        | .580        | .672        | .694        |
| Qwen3-Emb-8B       | Qwen3-Rank-8B       | .680        | .680        | .680        | .745        | .692        |
| SR-Emb-0.6B        | Qwen3-Rank-0.6B     | .720        | .693        | .707        | .769        | .724        |
| SR-Emb-0.6B        | Qwen3-Rank-8B       | .720        | .707        | .714        | .776        | .727        |
| <b>SR-Emb-0.6B</b> | <b>SR-Rank-0.6B</b> | <b>.760</b> | <b>.720</b> | <b>.740</b> | <b>.791</b> | <b>.704</b> |
| SR-Emb-8B          | SR-Rank-8B          | .787        | .733        | .760        | .808        | .719        |



## 5.2 主结果

**微调比单纯扩大规模更有价值。**表 2 显示，在纯编码器系统中，SR-Emb-0.6B 达到 65.4% 的平均 Hit@1，比同尺寸的 Qwen3-Emb-0.6B 基础模型提升 +9.4pp，并且在存在 13 倍参数差距的情况下仍以微弱优势胜过 64.0% 的 Qwen3-Emb-8B。这表明在该场景下，技能路由数据与任务专用负例可以补偿 13 倍的参数差距。

**检索器也为重排器提供了有用的提升空间。**SR-Emb-0.6B 达到 75.4% 的平均 R@20，超过 Qwen3-Emb-8B 的 72.6%。这一点很重要，因为只有当正确技能进入候选集时重排才能起作用。因此编码器的改进不仅是更好的 top-1 排序，也是为第二阶段提供更好的覆盖率。

**紧凑流水线以 1/13 的参数量匹敌或超越最强的基础系统。**表 3 显示，我们的主 1.2B 流水线 SR-Emb-0.6B  $\times$  SR-Rank-0.6B 达到 74.0% 的平均 Hit@1，相比之下 16B 最强基础流水线（Qwen3-Emb-8B  $\times$  Qwen3-Rank-8B）为 68.0%。它还比同尺寸的 1.2B 基础配置提升 +10.0pp，比使用相同经调优编码器的纯编码器检索提升 +8.6pp。该增益在 Easy (+8.0pp) 和 Hard (+4.0pp) 上都保持为正。结合第 5.6 节的服务结果——延迟降低 5.8 倍、GPU 显存减少 15.8%——紧凑流水线在效率-准确率前沿上占据了一个有利位置。第 5.4 节进一步验证了相同方向的优势在一个独立构建的补充基准上依然成立。

**基础重排器有帮助，但经调优的重排帮助更大。**经调优的 1.2B 流水线达到 74.0%，而 SR-Emb-0.6B  $\times$  Qwen3-Rank-8B 为 71.4%、16B 基础流水线为 68.0%，表明两个阶段的任务专用适配都对总体增益有贡献（附录 J 给出查询级分解）。LLM-as-judge 基线缺乏竞争力：最强的评判者（GPT-4o-mini (OpenAI, 2024a)）在相同候选名单上仅达到 67.3% 的 Hit@1，GPT-5.4-mini (OpenAI, 2026) 为 66.0%；两个评判者都只提供一个 top-1 选择，而非带分数的完整重排（附录 I）。相同的训练配方也可放大到 8B，得到 76.0% 的 Hit@1，不过 1.2B 系统已经捕获了大部分增益。

## 5.3 指标校准与关键消融

**Hit@1 增益应被解读为 top-1 路由增益。**表 4 显示主流线在单技能和多技能查询上都提升了 Hit@1。最强的基础流水线在严格的多技能 FC@10 上仍然更优（.382 对 .353），因此我们的主要论断是最强的 top-1 路由，而非全面更优的穷尽集合恢复。

**两项训练选择是必不可少的。**表 5 显示假阴性过滤为编码器贡献 +4.0pp 的 Hit@1 和 +3.1pp 的 MRR@10，而列表式训练对重排是决定性的：逐点变体崩塌至 43.3% 的 Hit@1，而列表式模型达到 74.0%。额外的覆盖率分析见表 4 和附录表 25。

表 4: 两个基础流水线与我们主 1.2B 流水线的单技能与多技能校准。Hit@1 反映 top-1 路由，R@10 反映候选名单覆盖率，FC@10 反映多技能查询的严格完整覆盖。

| 流水线                              | 单技能 Hit@1 | 单技能 R@10 | 单技能 FC@10 | 多技能 Hit@1 | 多技能 R@10 |
|----------------------------------|-----------|----------|-----------|-----------|----------|
| Qwen3-Emb-0.6B × Qwen3-Rank-0.6B | .625      | .708     | .708      | .647      | .556     |
| Qwen3-Emb-8B × Qwen3-Rank-8B     | .667      | .812     | .812      | .686      | .636     |
| SR-Emb-0.6B × SR-Rank-0.6B       | .729      | .875     | .875      | .745      | .624     |

表 5: 关键消融。假阴性过滤贡献 +4.0pp 的编码器 Hit@1；列表式重排是必不可少的，比逐点变体高出 +30.7pp。上：编码器变体。下：以 SR-Emb-0.6B 为检索器的重排器变体。

| 组件                              | 变体                     | Hit@1 | MRR@10 | R@10 |
|---------------------------------|------------------------|-------|--------|------|
| <b>编码器训练</b>                    |                        |       |        |      |
| SR-Emb-0.6B                     | 干净负例 (Clean negatives) | .653  | .723   | .688 |
| SR-Emb-0.6B                     | 原始负例 (Raw negatives)   | .613  | .692   | .672 |
| <b>重排器训练</b>                    |                        |       |        |      |
| SR-Emb-0.6B                     | 仅编码器 (无重排)             | .653  | .723   | .688 |
| SR-Emb-0.6B × Qwen3-Rank-0.6B   | 基础重排器                  | .707  | .769   | .724 |
| SR-Emb-0.6B × SR-Rank-0.6B (PW) | 逐点 BCE 微调              | .433  | .578   | .573 |
| SR-Emb-0.6B × SR-Rank-0.6B (LW) | 列表式 CE 微调              | .740  | .791   | .704 |

5.4 补充基准验证

为检验所观察到的增益是否能泛化到 75 条查询的核心基准之外，我们构建了 SkillBench-Supp，一个独立构建的补充基准，包含来自三个来源的 100 个 GT 技能，以及覆盖一个 77K 技能池的 256 条单标签评估查询。查询生成所用的 LLM、提示词设计和来源组合都与训练流水线不同；其中 30 个从池中选取的 GT 技能被显式地从训练中保留出去。在不重新调优、使用相同检查点的情况下，紧凑的 1.2B SkillRouter 流水线在 Hit@1 上以微弱优势胜过 16B 的 Qwen3 基础流水线 (.641 对.637)，且在 0.6B 和 8B 两个尺度上都呈现相同方向的优势。附录 M 提供了完整的构建细节、完整结果（附录表 26）和难度细分。

**表 6:** 在 75 任务核心集上、使用来自自然池（不含 Hard 分层干扰项）的技能进行的端到端智能体评估。结果在 3 次试验  $\times$  4 个编程智能体上取平均。黄金技能是 Oracle 上界。

| 技能条件 | 路由器/来源                              | Top-K | 单技能成功率 | 多技能成功率 | 总体成功率  |
|------|-------------------------------------|-------|--------|--------|--------|
| 无技能  | 无                                   | —     | 12.50% | 16.01% | 14.89% |
| 黄金技能 | Oracle 黄金标准                         | GT    | 30.90% | 33.50% | 32.67% |
| 检索技能 | Qwen3-Emb-8B $\times$ Qwen3-Rank-8B | 1     | 26.74% | 25.33% | 25.78% |
| 检索技能 | SR-Emb-0.6B $\times$ SR-Rank-0.6B   | 1     | 29.86% | 26.47% | 27.56% |
| 检索技能 | Qwen3-Emb-8B $\times$ Qwen3-Rank-8B | 10    | 20.49% | 27.78% | 25.45% |
| 检索技能 | SR-Emb-0.6B $\times$ SR-Rank-0.6B   | 10    | 26.04% | 28.60% | 27.78% |

## 5.5 下游端到端智能体评估

**路由增益迁移到直接的智能体执行。**四个编程智能体——Kimi-K2.5 (Kimi Team, 2026)、glm-5 (Z.AI, 2026)、Claude Sonnet 4.6 和 Claude Opus 4.6 (Anthropic, 2026) ——在 Claude Code 框架 (Anthropic, 2025) 内运行，超时设为 1200s；该框架将每个被检索技能的名称和描述注入智能体上下文，任务设置与成功判定标准遵循 SkillsBench (Li et al., 2026b)。如表 6 所示，在 top-1 和 top-10 两种设置下，SkillRouter 都相对最强基础路由器提升了平均任务成功率（分别为 +1.78pp 和 +2.33pp），相比基础路由器恢复了「无技能  $\rightarrow$  黄金技能」提升幅度的约 71–73%（基础路由器为 59–61%）。top-1 和 top-10 产生相似的总体成功率（约 27.6–27.8%），表明在候选名单扩展超过某一质量阈值后回报递减。对更强的智能体而言增益更明显：Claude Sonnet/Opus 4.6 平均增益为 +3.22pp，而 glm-5 和 Kimi-K2.5 为 +0.89pp，这与「对于无法充分利用正确路由技能智能体，路由效用存在天花板」的解释一致。附录 L 报告了各智能体的细分；附录 L.1 给出代表性案例。

## 5.6 服务效率

在一个真实池的 GPU 服务基准上，对 80 条计时查询（274–5109 字符），1.2B SkillRouter 流水线以 495.8ms 的中位延迟服务在线查询路径——比 16B 基础流水线快 5.8 倍，同时使用的 GPU 显存少 15.8%（附录 H）。

# 6 相关工作

LLM 智能体依赖于大规模的工具与技能集合。已有工作研究了从小型固定工具集到大型 API 仓库等各种场景下的工具调用与检索 (Schick et al., 2023; Shen et al., 2023; Qin et al., 2023; Patil et al., 2023; Du et al., 2024; Yuan et al., 2024; Zheng et al., 2024)。然而，这些系统通常从小得多的池中检索、强调工具使用而非上游路由，或主要在元数据上运作。我们的场景针对的是现代技能生态所造成的挑战：大规模、高度重叠的注册库，在其中许多候选都可能对同一个查询显得相关。

我们的系统设计遵循神经信息检索 (neural IR) 中标准的「检索-重排」范式 (Karpukhin et al., 2020; Izacard et al., 2022; Xiong et al., 2021; Wang et al., 2022; Li et al., 2023; Xiao et al., 2024; Nogueira & Cho, 2019; Sun et al., 2023), 但我们的场景在两方面有所不同: 技能是带有严重技能间同质性的结构化多字段对象, 且我们的评估明确研究了当模型能访问完整主体 (而非仅名称和描述) 时路由质量如何变化。我们的基准建立在 SkillsBench (Li et al., 2026b) 之上, 但将焦点从下游工具使用转移到大规模技能检索。因此从方法论上看, 我们的贡献并非一种新的重排架构, 而是一个为同质技能池 (在其中假阴性与列表式竞争占主导) 量身定制的端到端全文本路由配方与基准设置。

## 7 结论

我们在真实的注册库规模下研究技能路由, 并表明完整的技能文本是一个关键的路由信号: 在我们约 80K 技能的基准上, 移除主体文本会导致代表性基线下降 31–44pp。一个紧凑的 1.2B 全文本「检索-重排」流水线达到 74.0% 的 Hit@1——在延迟降低 5.8 倍的情况下与最强的 16B 基础流水线相竞争——其中假阴性过滤和列表式重排损失被证明在同质技能池中是必不可少的。这些增益泛化到了一个补充基准 (§5.4), 并迁移到了跨四个编程智能体的直接任务执行中。更广泛地说, 主体访问这一发现很可能延伸到技能路由之外、其他具有丰富实现细节的结构化检索场景, 例如 API 路由和插件选择。

**局限性。**我们的基准源自数量有限的来源; 相关论断适用于大规模、高度重叠的注册库, 而在较小的目录中仅元数据路由可能更具竞争力。下游评估仅限于单一执行预算下的四个编程智能体; FC@10 与端到端 top-10 成功率度量的是不同的量 (穷尽式黄金集合恢复 vs. 有界打包效用), 不应被视为可互换。

## 参考文献

1. Anthropic. Claude Code overview. <https://code.claude.com/docs>, 2025.
2. Anthropic. Models overview. <https://platform.claude.com/docs/en/about-claude/models/overview>, 2026.
3. Yu Du, Fangyun Wei, and Hongyang Zhang. Anytool: Self-reflective, hierarchical agents for large-scale api calls. In Proceedings of the 41st International Conference on Machine Learning, 2024. <https://openreview.net/forum?id=qFILbkTQWw>.
4. Google. Gemini embedding model. <https://ai.google.dev/gemini-api/docs/embeddings>, 2025.
5. Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, and Lichao Sun. Metatool benchmark for large language models: Deciding whether to use tools and which to use. arXiv preprint arXiv:2310.03128, 2024.
6. Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand

- Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.
7. Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.
  8. Kimi Team. Kimi k2.5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2602.02276*, 2026. *arXiv:2602.02276*.
  9. Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nv-embed: Improved techniques for training llms as generalist embedding models. *arXiv preprint arXiv:2405.17428*, 2024. *ICLR 2025 Spotlight*.
  10. Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*, 2026a.
  11. Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han-chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026b. *arXiv:2602.12670*.
  12. Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023.
  13. Majiayu000. Claude Skill Registry Core. <https://github.com/majiayu000/claude-skill-registry-core>, 2026. GitHub repository; accessed March 30, 2026.
  14. Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.
  15. OpenAI. Gpt-4o mini model. <https://platform.openai.com/docs/models/gpt-4o-mini>, 2024a.
  16. OpenAI. New embedding models and api updates. <https://openai.com/blog/new-embedding-models-and-api-updates>, 2024b.
  17. OpenAI. Codex — AI coding partner from OpenAI. <https://openai.com/codex/>, 2025.
  18. OpenAI. Gpt-5.4 mini model. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>, 2026.

19. OpenClaw. OpenClaw built-in skills. <https://github.com/openclaw/openclaw/tree/main/skills>, 2026a. Official skill repository; accessed March 2026.
20. OpenClaw. Skills - OpenClaw. <https://docs.openclaw.ai/tools/skills>, 2026b.
21. Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. arXiv preprint arXiv:2305.15334, 2023.
22. Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. arXiv preprint arXiv:2307.16789, 2023.
23. Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
24. Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761, 2023.
25. Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 2023.
26. Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is chatgpt good at search? investigating large language models as re-ranking agents. arXiv preprint arXiv:2304.09542, 2023.
27. Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. arXiv preprint arXiv:2212.03533, 2022.
28. Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. arXiv:2309.07597.
29. Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *Proceedings of ICLR*, 2021.
30. Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R. Fung, Hao Peng, and Heng Ji. Craft: Customizing llms by creating and retrieving from specialized toolsets. In *Proceedings of ICLR*, 2024.
31. Z.AI. Chat completion - overview. <https://docs.z.ai/api-reference/llm/chat-completion>, 2026.

32. Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. arXiv preprint arXiv:2506.05176, 2025. <https://qwenlm.github.io/blog/qwen3-embedding/>.
33. Yuanhang Zheng, Peng Li, Wei Liu, Yang Liu, Jian Luan, and Bin Wang. Toolrerank: Adaptive and hierarchy-aware reranking for tool retrieval. In Proceedings of LREC-COLING, pp. 16263–16273, 2024.

## A 评估细节

**单技能 vs. 多技能查询。**在 75 条核心查询中，24 条是单技能查询（恰好一个黄金标准技能），51 条是多技能查询（需要两个或更多黄金标准技能才能完成任务）。

**多技能查询的指标计算。**对于具有多个黄金标准技能  $G_q = \{g_1, \dots, g_m\}$  的查询，我们将 Hit@1 定义为「是否有任意一个黄金标准技能出现在第 1 名」的指示量。MRR@10 使用排名最高的黄金标准技能的倒数排名。Recall@K 度量出现在前 K 个结果中的黄金标准技能所占比例： $R@K = |G_q \cap \text{top-}K| / |G_q|$ 。FC@10（Full Coverage at 10，前 10 名完整覆盖率）是最严格的指标：仅当某查询的所有黄金标准技能都出现在前 10 名时它才等于 1。因此我们用 R@10 和 FC@10 来补充这一「命中任一即可」的判据。

**核心查询选择协议。**底层的相关性文件包含 87 个由 SkillsBench 派生的任务。我们将所报告的 75 条核心查询基准定义为「至少含一个非通用核心技能（core\_gt\_ids 非空）」的子集，这产生了 59 个干净任务和 16 个混合任务。其余 12 个任务为仅通用任务：它们的标签只包含辅助性的文件类型技能，如 pdf、docx、pptx 或 xlsx，因此被排除在分层特定的路由指标之外。

**80K 池中的元数据丰富度。**较差的 nd 结果不能仅用缺失描述来解释。在 Easy 池中，描述几乎总是存在（仅 0.12% 为空），但它们比完整的技能主体短得多：描述长度中位数为 21 词，而主体为 704 词。表 7 总结了由此得到的字段长度分布。

表 7: 78,361 技能 Easy 池的元数据审计。描述通常存在，但比主体压缩得多。

| 统计量       | 值       |
|-----------|---------|
| 描述为空      | 0.12%   |
| 描述 < 10 词 | 18.66%  |
| 描述 < 25 词 | 59.22%  |
| 描述长度中位数   | 21 词    |
| 主体长度中位数   | 704 词   |
| 主体长度 P90  | 1,991 词 |



**查询多样性。**表 8 总结了 75 条核心查询的多样性概况。这些查询跨越 55 个应用领域，为便于阅读我们将其归为 8 个超级类别；分布相对均衡，没有任何单一超级类别超过 17% 的查询。在这些领域内，244 个独特的主题标签覆盖了从地震学、量子模拟到 BGP 路由、视频配音等领域。难度分布有意偏向非平凡任务（中等 60%、困难 35%、简单 5%），与本基准聚焦于大池路由（其中简单案例信息量较少）的定位一致。在 51 条多技能查询中，先前的分析识别出三种结构类型：互补/流水线任务（43%），其中技能构成顺序阶段；替代/重叠任务（25%），其中多个技能服务于相似功能；以及兼具两种模式的混合任务（32%）。这种结构差异锻炼了路由质量的不同方面：流水线查询考验召回（所有阶段都必须被找到），而替代查询考验精确率（最佳备选项必须被排在第一）。

表 8: 75 条核心基准查询的多样性概况。

| 维度         | 值                                         |
|------------|-------------------------------------------|
| 应用领域       | 55（8 个超级类别）                               |
| 独特主题标签     | 244                                       |
| 难度划分       | easy 4 / medium 45 / hard 26              |
| 单技能/多技能    | 24 / 51                                   |
| 每查询 GT 技能数 | 1–7（均值 2.75）                              |
| 多技能类型      | pipeline 43% / substitute 25% / mixed 32% |
| 指令长度       | 36–586 词（中位数 169）                         |

**主体访问研究的纯编码器 nd/full 细节。**表 9 报告了第 3 节中引用的完整纯编码器 nd/full 比较。所有数字都使用与正文相同的基准池和相同的 75 条核心查询。Easy 和 Hard 是主论文中报告的两个评估分层。

B 基准数据

**Hard 分层干扰项生成。**Hard 分层用 780 个由 LLM 生成的干扰技能扩充了 Easy 池。对于每个黄金标准技能，我们提示 GPT-4o-mini 使用三种干扰策略生成 3–5 个看似合理但不正确的技能：同领域-不同问题（同一技术领域但解决不同任务）、同技术-不同用途（同一技术栈但不同应用）以及过度泛化（缺乏所需特定能力的更宽泛版本）。我们将这些干扰项用作针对功能级混淆的定向鲁棒性压力测试，而非对它们在自然仓库中确切普遍程度的估计。表 10 展示了生成提示词。

**池去重与规范化。**80K 池仅按技能 ID 去重。完全重复的 ID 作为数据清洗产物被移除，但与黄金标准技能同名的重叠被有意保留，以保留真实的近重复混淆，而非构造一个人为无冲突的目录。

**数据样例。**表 11 展示了来自 Easy 和 Hard 分层的代表性样例，说明了黄金标准技能、池内技能（自然干扰项）与 LLM 生成干扰项之间的差异。

## C 详细注意力分析

表 12 总结了 SR-Rank-0.6B 的原始逐层注意力分布。由于原始注意力质量受长度混淆，表 13 将相同的轨迹与字段 token 占比基线进行比较。图 3 随后给出对同一证据最有用的两种可视视图：一个针对 token 占比基线的逐层轨迹，以及一个查询级的最终层检查。

**表 9:** 在与正文相同的基准池和 75 条核心查询上的纯编码器 nd/full 细节。Easy 和 Hard 是主论文中报告的两个评估分层。

| 模型             | 输入   | Easy Hit@1 | Hard Hit@1 | Avg Hit@1 |
|----------------|------|------------|------------|-----------|
| BM25           | nd   | .000       | .000       | .000      |
| BM25           | full | .347       | .280       | .314      |
| Qwen3-Emb-0.6B | nd   | .227       | .147       | .187      |
| Qwen3-Emb-0.6B | full | .587       | .533       | .560      |
| Qwen3-Emb-8B   | nd   | .307       | .200       | .253      |
| Qwen3-Emb-8B   | full | .653       | .627       | .640      |

**表 10:** Hard 评估分层的干扰技能生成提示词。

**System (系统提示):** You are a skill document writer for a coding agent platform. You produce SKILL.md-style documents that are plausible but address a DIFFERENT problem than the reference skill. Each distractor must look like a real, useful skill document but must NOT solve the same task as the reference.

**User (用户):** I have a ground-truth skill used for the task(s): <task ids>. Reference skill (name: <skill name>, category: <category>): <body truncated>. Generate <num distractors> HARD distractor skills. Each distractor must be a complete SKILL.md document that looks relevant to someone searching for this skill, but actually solves a different problem. Use these distractor strategies (one per distractor): same-domain-diff-problem, same-tech-diff-use, over-generalized. For EACH distractor, output a JSON object with fields: distractor type, name, description, body (400–1200 words).

**精确计算。**对于每个被分析的「查询-技能」对  $i$ ，我们对扁平化的重排器输入进行分词，并识别出三个技能字段  $f \in \{\text{name, desc, body}\}$  在模型输入中的 token 跨度  $T_{i,f}$ 。设  $p_i$  表示最后一个输入位置，其隐藏状态用于预测下一个 token，从而做出 yes/no 决策。在每一层  $\ell$ ，我们对各注意力头上从  $p_i$  出发的注意力取平均，并将所得质量在每个字段的 token 上求和。然后我们在三个技能字段内归一

化：

$$a_{i,\ell,f} = \frac{\sum_{t \in T_{i,f}} \bar{A}_{i,\ell}(p_i, t)}{\sum_{f'} \sum_{t \in T_{i,f'}} \bar{A}_{i,\ell}(p_i, t)}.$$

对应的长度基线是该字段的 token 占比：

$$b_{i,f} = \frac{|T_{i,f}|}{\sum_{f'} |T_{i,f'}|}.$$

如果注意力主要由字段长度解释，我们会预期  $a_{i,\ell,f}$  在各层中保持接近  $b_{i,f}$ 。图 3（左）将逐层的平均注意力占比与平均 token 占比基线进行比较，而图 3（右）将最终层的主体注意力  $a_{i,L,\text{body}}$  与每查询的基线  $b_{i,\text{body}}$  进行比较。

**表 11：**来自 Easy 和 Hard 分层的代表性数据样例。黄金标准技能出现在两个分层中；LLM 生成干扰项仅在 Hard 分层中加入，以增加技能间混淆。

| 类型                                    | 名称                  | 描述                | 主体（截断）                                                             |
|---------------------------------------|---------------------|-------------------|--------------------------------------------------------------------|
| <b>黄金标准技能</b> （Easy 和 Hard 中均存在）      |                     |                   |                                                                    |
| GT                                    | speech-to-text      | 使用 Whisper 转录音频文件 | 使用 OpenAI Whisper 模型将音频/视频文件转换为文本。支持长文件的分块处理、多种输出格式（txt、srt、vtt）…… |
| <b>自然池技能</b> （Easy 和 Hard 中均存在——来自社区） |                     |                   |                                                                    |
| Pool                                  | audio-transcriber   | 音频转录服务            | 一个使用云端 API 的通用音频转录技能。将音频发送至外部服务，返回带时间戳的 JSON 转录……                  |
| <b>LLM 生成干扰项</b> （仅在 Hard 分层中加入）      |                     |                   |                                                                    |
| Distractor                            | video-subtitle-sync | 将字幕与视频同步          | 调整字幕时间以匹配视频播放。解析 SRT 文件，检测音频线索以对齐，处理帧率转换……                         |

表 12: SR-Rank-0.6B (28 层 × 16 头, 75 条查询) 按层分组与关键单层的各技能字段原始注意力分布。

| 层 (分组)     | Name  | Desc | Body  |
|------------|-------|------|-------|
| 层分组 (取平均)  |       |      |       |
| 早期 (0–6)   | 2.3%  | 0.3% | 97.3% |
| 中期 (7–20)  | 9.6%  | 1.4% | 89.0% |
| 后期 (21–27) | 7.5%  | 0.8% | 91.7% |
| 关键单层       |       |      |       |
| 第 0 层      | 0.3%  | 0.1% | 99.6% |
| 第 11 层     | 14.6% | 1.4% | 84.0% |
| 第 19 层     | 26.3% | 1.8% | 72.0% |
| 第 27 层     | 1.5%  | 0.4% | 98.1% |
| 总体         | 7.3%  | 1.0% | 91.7% |

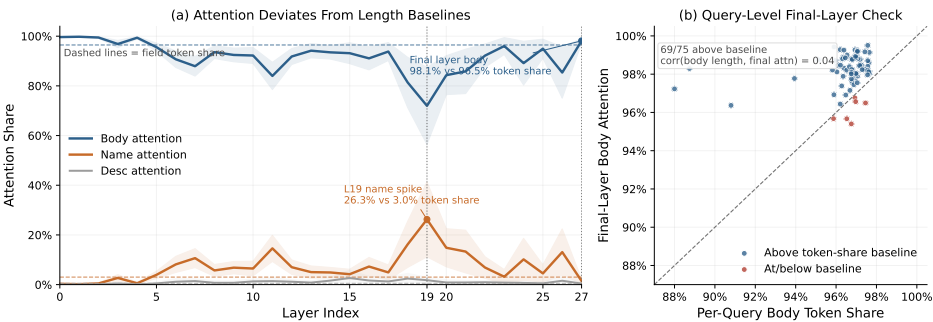


图 3: SR-Rank-0.6B 在 75 个「查询-技能」对上的长度受控注意力可视化。左：逐层平均注意力轨迹与各字段的 token 占比基线对比；阴影带表示 75 个「查询-技能」对上 ±1 标准差。短小的名称字段在中间层远高于其 3.0% 的 token 占比基线，而最终层回落到主体。右：查询级的最终层主体注意力与各查询的主体 token 基线对比。大多数点位于对角线之上，意味着最终层对主体的关注超过了纯长度基线所能预测的程度。

表 13: 相同 75 个「查询-技能」对的长度受控注意力诊断。注意力占比从用于预测下一个 token（从而做出 yes/no 决策）的最后一个输入位置度量，在各头上取平均，并在 name/description/body 字段内归一化。如果注意力主要由字段长度解释，逐层注意力将保持接近字段 token 占比。

| 字段   | Token 占比 | 总体注意力 | 第 19 层 | 第 27 层 |
|------|----------|-------|--------|--------|
| Name | 3.0%     | 7.3%  | 26.3%  | 1.5%   |
| Desc | 0.5%     | 1.0%  | 1.8%   | 0.4%   |
| Body | 96.5%    | 91.7% | 72.0%  | 98.1%  |

**为什么这不只是「更多文本」。**表 12 中 91.7% 的总体主体注意力应相对于以下事实来解读：在这些被分析的对中，主体字段本就占据了 96.5% 的技能 token。因此一个纯粹的「更长文本」解释会预测注意力在各层中保持接近 token 占比基线。然而，表 12-13 和图 3 显示了一个结构化的「主体 → 名称 → 主体」轨迹：短小的名称字段仅占 3.0% 的 token，却在第 19 层获得 26.3% 的注意力，而最终层回落到 98.1% 的主体注意力。因此解读图 3 最简单的方式是：左图表明注意力在整个网络中并不简单地跟随字段长度，而右图表明这种最终层向主体的回归并非由少数离群查询驱动。在查询层面，最终层主体注意力在 75 条查询中的 69 条上超过了主体的 token 占比，且与绝对主体长度几乎不相关 ( $r = 0.04$ )。这些诊断反对一个平凡的长度效应，并与以下解释一致：重排器使用名称进行中间对齐，使用主体进行最终的相关性判定——尽管它们并不意在作为一个完全因果的检验。

## D 描述质量分层

一个自然的担忧是，第 3 节报告的 nd→full 差距可能由较差的 GT 技能描述驱动：如果描述更详尽，仅元数据路由或许就足够了。为检验这一点，我们按 GT 技能描述的词数对 148 条匹配的 Easy+Hard 查询进行分层，将 188 个具有可提取描述的 GT 技能划分为四个四分位 (Q1 ≤19 词, Q2 20–27, Q3 28–35, Q4 >35)。表 14 报告了最强基础编码器 (Qwen3-Emb-8B) 在约 80K 池上各四分位的 nd vs. full Hit@1。

**表 14:** 按 GT 描述词数分层的 nd vs. full Hit@1。该差距在每个四分位中都保持很大 (≥26pp)，包括描述最长的 Q4。基线：Qwen3-Emb-8B，约 80K 池，Easy+Hard 取平均。

| 四分位          | N   | Full Hit@1 | ND Hit@1 | 差距 (pp) |
|--------------|-----|------------|----------|---------|
| Q1 (≤19 词)   | 26  | 53.8%      | 26.9%    | +26.9   |
| Q2 (20–27 词) | 40  | 80.0%      | 30.0%    | +50.0   |
| Q3 (28–35 词) | 38  | 65.8%      | 26.3%    | +39.5   |
| Q4 (>35 词)   | 44  | 52.3%      | 20.5%    | +31.8   |
| 总体           | 148 | 63.5%      | 25.7%    | +37.8   |

该差距并不随描述长度单调减小：Q4（描述最长，>35 词）仍呈现 +31.8pp 的差距，而 Q2 表现出最大的差距 (+50.0pp)。这一模式反对描述质量混淆因素的解释，并支持以下解读：主体文本提供了描述——无论其长度或细节如何——都无法替代的路由信号。

## E 训练数据构建

本附录提供 SkillRouter 流水线两个阶段训练数据构建的完整细节，补充第 4 节中的总结。

## E.1 查询生成

第 4 节总结了 37,979 个合成的（查询，技能）训练对；这里我们提供生成细节。技能从 51 个类别中分层采样。生成的查询平均长度为 160 词。表 15 展示了与 GPT-4o-mini 一起使用的提示词模板。

**表 15:** 查询生成提示词模板（简化版）。LLM 接收技能的元数据，并在不提及技能名称的情况下生成一个真实的任务描述。

---

**System (系统提示):** You are an experienced user of AI assistants. You write clear, realistic task requests that describe what you need to accomplish.

**User (用户):** Given the following skill specification, write a realistic task description that someone would ask an AI assistant to help with. The task should naturally require the capabilities described in this skill. Skill name: <name>. Category: <category>. Description: <description>. Skill body: <body preview>. Requirements: (1) Describe a concrete scenario with specific inputs/outputs. (2) Include enough detail that the skill would be clearly useful. (3) Do NOT mention the skill name “<name>” anywhere in the task. Output ONLY the task description.

---

## E.2 困难负例挖掘

有效的对比学习需要有信息量的负例——它们具有挑战性但不是假正例。我们采用一种多来源挖掘策略，为每个查询产生来自四个互补来源的 10 个负例：

- **语义负例（每查询 4 个）:** 我们使用基础 Qwen3-Emb-0.6B 模型为所有技能预计算嵌入，按余弦相似度检索 top-50 最相似技能，并从该集合中采样 4 个非正例技能。这些是最困难的负例——语义接近但功能不同。
- **词法负例（每查询 3 个）:** 在同一 top-50 候选集上对技能文本（名称 + 描述 + 主体）进行 BM25 评分，捕捉语义搜索可能遗漏的词项重叠混淆项。
- **分类法负例（每查询 2 个）:** 从与正例技能同一类别但名称不同的技能中随机采样，提供同领域干扰项。
- **随机负例（每查询 1 个）:** 从不同类别中均匀采样，作为用于校准的简单负例。

## E.3 假阴性过滤

如第 4 节所述，被挖掘的负例经过一个三层过滤器的后处理。各层的移除数量为：

1. **名称去重:** 移除与该查询任何黄金标准技能同名的负例（移除 24,879 对）。

2. **主体重叠**: 移除其主体文本与某黄金标准技能主体的 trigram Jaccard 相似度  $> 0.6$  的负例 (移除 13,860 对)。
3. **嵌入相似度**: 移除与某黄金标准技能嵌入的余弦相似度  $> 0.92$  的负例, 捕捉词法匹配遗漏的语义重复 (移除 326 对)。

总计移除 39,065 对假阴性 (约占所有被挖掘负例对的 10%)。

## E.4 重排器训练数据

对于 32,283 条训练查询中的每一条, 我们使用训练好的 SR-Emb-0.6B 编码器检索 top-20 候选。每个候选名单包含 20 个带二元相关性标签 (正例或负例) 的技能, 构成列表式交叉熵优化的一个训练组。上述相同的三层假阴性过滤也应用于重排器训练数据。

## F 模型输入模板与训练细节

我们记录所报告模型使用的精确输入格式、字段截断规则、损失函数以及选定的训练设置。

**双编码器 (查询侧)**。遵循 Qwen3-Emb 的指令前缀编码格式:

```
Instruct: Given a task description, retrieve the most relevant skill document that
would help an agent complete the task
Query: <query text>
```

在分词之前, 原始查询文本被截断到 1,500 个字符。

**双编码器 (技能侧)**。技能被编码为不带指令前缀的纯拼接文本:

```
<name> | <description> | <body>
```

在分词之前, 描述被截断到 300 字符、主体被截断到 2,500 字符。在编码器训练期间, 每个查询/正例/负例输入以最大长度 2,048 token 进行分词。

**交叉编码器重排器 (扁平化全文本格式)**。遵循 Qwen3-Rank 的输入约定 (Zhang et al., 2025):

```
<Instruct>: Given a task description, judge whether the skill document is relevant
and useful for completing the task
<Query>: <query text>
<Document>: <name> | <description> | <body>
```

在构造提示词之前, 描述被截断到 500 字符、主体被截断到 2,000 字符。分词后的重排器输入使用最大长度 4,096 token。

**LLM-as-judge (GPT-4o-mini / GPT-5.4-mini)**。我们评估 OpenAI GPT-4o-mini 和 GPT-5.4-mini (OpenAI, 2024a; 2026) 作为列表式评判者。两个 LLM 评判者都以列表式模式运作：它们一次性接收完整的 top-K 候选列表，并选出单个最相关的技能。

系统提示词：

You are an expert at matching tasks to reusable skill definitions. Given a task query and a numbered list of candidate skills, identify the SINGLE most relevant skill that best solves the task. Respond with ONLY the number (e.g. '3') of the best matching skill, nothing else.

**表 16:** 所报告 SkillRouter 模型的选定训练超参数。字符上限在分词之前应用。

| 设置        | SR-Emb-0.6B                            | SR-Rank-0.6B                 |
|-----------|----------------------------------------|------------------------------|
| 目标        | In-batch InfoNCE ( $\tau=0.05$ )       | Listwise CE ( $\tau=1.0$ )   |
| 输入/提示     | 查询指令前缀；技能 = name   description   body  | 在 top-20 候选上的扁平化全文本「查询-文档」提示 |
| 字段上限      | query 1500 字符；desc 300 字符；body 2500 字符 | desc 500 字符；body 2000 字符     |
| 最大长度      | 2048                                   | 4096                         |
| Epoch     | 1                                      | 1                            |
| Batch     | 8                                      | 1 个列表式组                      |
| GA (梯度累积) | 4                                      | 16                           |
| LR (学习率)  | $2 \times 10^{-5}$                     | $1 \times 10^{-5}$           |

用户消息：查询文本，后跟一个编号的候选列表，每个候选格式化为：

```
[1] Name: <name>
Description: <description>
Body: <body>
[2] Name: ...
```

被选中的技能被置于第 1 名；所有其他候选保留其原始的编码器排序。对于 LLM-评判者实验，每个候选使用与重排器相同的字段上限：在构造提示词之前，描述被截断到 500 字符、主体被截断到 2,000 字符。

**损失定义。**所报告的 SR-Emb-0.6B 模型使用 in-batch InfoNCE：

$$\mathcal{L}_{enc} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\text{sim}(q_i, s_i^+)/\tau)}{\sum_j \exp(\text{sim}(q_i, s_j)/\tau)} \quad (1)$$

其中  $\tau = 0.05$ ， $\text{sim}(\cdot, \cdot)$  是归一化嵌入上的余弦相似度。



所报告的 SR-Rank-0.6B 模型在 top-K 候选集上使用列表式交叉熵：

$$\mathcal{L}_{LW} = -\log \frac{\exp(f(q, s^+)/\tau)}{\sum_{j=1}^K \exp(f(q, s_j)/\tau)} \quad (2)$$

其中  $f(q, s)$  是重排器分数，训练中  $\tau = 1.0$ 。仅对于逐点消融，我们改用二元交叉熵：

$$\mathcal{L}_{PW} = -\frac{1}{K} \sum_{j=1}^K [y_j \log \sigma(f_j) + (1 - y_j) \log(1 - \sigma(f_j))]. \quad (3)$$

**训练超参数。**所有报告的训练运行都使用单块具有 96GB HBM3 显存的 NVIDIA GPU (Hopper 架构)。除非另有说明，两个报告的模型都使用权重衰减为 0.01 的 AdamW、带 5% 预热 (warmup) 的余弦学习率调度、BF16 混合精度以及梯度检查点 (gradient checkpointing)。表 16 总结了所报告模型的选定训练设置，且两个报告的模型都使用 1-epoch 训练配置来描述。

## G Top-K 候选消融

对于主基准的工作点，我们对从编码器传给重排器的候选数量 ( $K \in \{10, 20, 50\}$ ) 进行消融。表 17 报告了三个重排器在两个分层上的 Hit@1。图 4 展示了三个编码器检索器的 Recall@K 候选覆盖率，其中  $K=20$  处的星号标记表示对应的端到端流水线 Hit@1。

**表 17:** 在主基准上 Hit@1 随重排 top-K 候选数量的变化。编码器 = SR-Emb-0.6B。在所报告的 Easy/Hard 比较中， $K=20$  始终匹配或超过其他选择。

| 重排器               | Easy @10 | Easy @20 | Easy @50 | Hard @10 | Hard @20 | Hard @50 | Avg @20 |
|-------------------|----------|----------|----------|----------|----------|----------|---------|
| SR-Rank-0.6B (FT) | .747     | .760     | .733     | .720     | .720     | .707     | .740    |
| Qwen3-Rank-0.6B   | .720     | .720     | .667     | .693     | .693     | .640     | .707    |
| Qwen3-Rank-8B     | .693     | .720     | .707     | .680     | .707     | .707     | .714    |

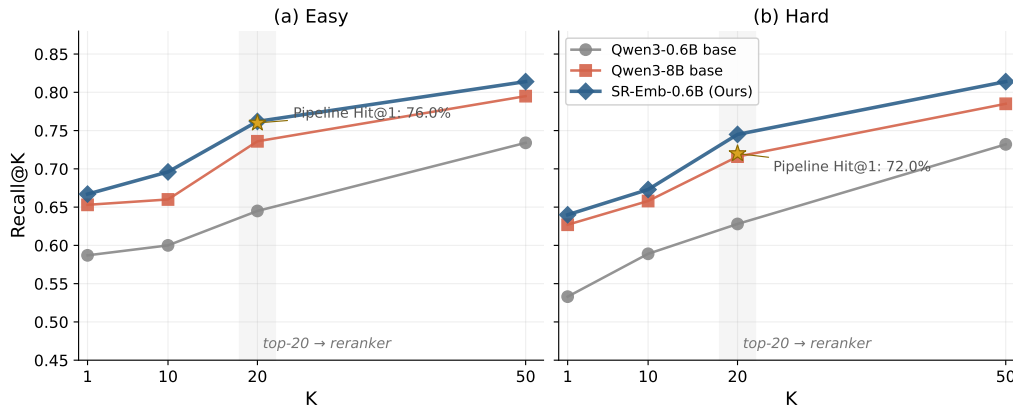


图 4: 三个编码器检索器在 Easy 和 Hard 上的 Recall@K 候选覆盖率。 $K=20$  处的星号标记表示主 SR-Emb-0.6B  $\times$  SR-Rank-0.6B 流水线在主工作点的端到端 Hit@1, 仅作为对照 Recall@K 曲线的参考显示。

综合来看, 表 17 和图 4 论证了我们为主基准工作点选择  $K=20$  的理由。图 4 表明  $K=20$  已经为所有三个检索器捕获了大部分可用的候选覆盖率。表 17 随后表明, 在这一主基准设置下, 转向  $K=50$  并不改善下游 Hit@1, 且常常使其变差, 尤其是对于经微调的 SR-Rank-0.6B (平均  $-2.0\text{pp}$ ), 而  $K=10$  则留下较少的重排提升空间。

## H 服务效率

表 18 报告了真实池上的 GPU 服务基准。这些度量仅涵盖在线查询路径: 一次编码器前向传递、近似最近邻检索, 以及 (适用时) top-20 重排。它们排除了一次性的模型加载、离线池嵌入和索引构建成本。

表 18: 80 条计时查询 (274–5109 字符) 上的真实池 GPU 服务基准。纯编码器行受编码器约束; 流水线行受重排前向约束。GPU-秒 / 1K 是一个基准计算占用度量, 而非美元成本估计。

| 系统                   | p50 (ms) | p95 (ms) | QPS  | GPU 显存 | GPU-秒 / 1K |
|----------------------|----------|----------|------|--------|------------|
| SR-Emb-0.6B          | 19.8     | 20.8     | 50.5 | 9364   | 19.8       |
| SR pipeline (1.2B)   | 495.8    | 871.4    | 1.83 | 18976  | 547.0      |
| Qwen3-Emb-8B         | 60.4     | 81.3     | 18.7 | 22539  | 53.5       |
| Qwen3 pipeline (16B) | 2900.1   | 5676.5   | 0.32 | 22539  | 6189.9     |

I 扩展的正文表格

表 19 和表 20 提供了按分层及汇总的完整编码器检索网格，而表 21 给出了完整的端到端结果。表 24 报告了分层鲁棒性总结，表 25 给出了完整的多指标重排器损失消融。

**表 19:** 80K 技能池上的完整编码器检索结果：Easy 和 Hard 分层指标。所有模型都使用完整技能文本作为输入。

| 模型                     | 类型      | 参数量  | E-Hit@1 | E-R@20 | H-Hit@1 | H-R@20 |
|------------------------|---------|------|---------|--------|---------|--------|
| BM25                   | Sparse  | –    | .347    | .376   | .280    | .354   |
| gemini-embedding-001   | Propri. | –    | .613    | .689   | .560    | .685   |
| text-embedding-3-large | Propri. | –    | .640    | .676   | .600    | .652   |
| E5-Large-v2            | Encoder | 335M | .507    | .594   | .493    | .594   |
| BGE-Large-v1.5         | Encoder | 335M | .613    | .677   | .587    | .658   |
| GTE-Large-v1.5         | Encoder | 434M | .573    | .706   | .520    | .686   |
| Qwen3-Emb-0.6B         | Decoder | 0.6B | .587    | .645   | .533    | .628   |
| NV-Embed-v2            | Decoder | 7B   | .440    | .565   | .413    | .559   |
| Qwen3-Emb-8B           | Decoder | 8B   | .653    | .736   | .627    | .716   |
| SR-Emb-0.6B            | Decoder | 0.6B | .667    | .762   | .640    | .745   |
| SR-Emb-8B              | Decoder | 8B   | .693    | .785   | .667    | .769   |

**表 20:** 80K 技能池上的完整编码器检索结果: Easy 和 Hard 上的平均指标。所有模型都使用完整技能文本作为输入。

| 模型                     | 类型      | 参数量  | Hit@1 | MRR@10 | R@10 | R@20 | R@50 |
|------------------------|---------|------|-------|--------|------|------|------|
| BM25                   | Sparse  | —    | .314  | .365   | .321 | .365 | .433 |
| gemini-embedding-001   | Propri. | —    | .587  | .650   | .629 | .687 | .774 |
| text-embedding-3-large | Propri. | —    | .620  | .658   | .609 | .664 | .709 |
| E5-Large-v2            | Encoder | 335M | .500  | .565   | .553 | .594 | .622 |
| BGE-Large-v1.5         | Encoder | 335M | .600  | .653   | .608 | .668 | .743 |
| GTE-Large-v1.5         | Encoder | 434M | .547  | .631   | .630 | .696 | .750 |
| Qwen3-Emb-0.6B         | Decoder | 0.6B | .560  | .638   | .595 | .637 | .733 |
| NV-Embed-v2            | Decoder | 7B   | .427  | .508   | .504 | .562 | .649 |
| Qwen3-Emb-8B           | Decoder | 8B   | .640  | .698   | .659 | .726 | .790 |
| SR-Emb-0.6B            | Decoder | 0.6B | .654  | .723   | .688 | .754 | .814 |
| SR-Emb-8B              | Decoder | 8B   | .680  | .731   | .692 | .777 | .851 |

**表 21:** 完整的编码器  $\times$  重排器结果 (top-20 候选)。分组标题指示了输入格式, 因此省略了单独的 rank 输入列。LLM 评判者只提供一个 top-1 选择, 故这些行只报告 Hit@1。

| 编码器                                | 重排器             | E-Hit@1 | H-Hit@1 | A-Hit@1 | A-MRR@10 | A-R@10 |
|------------------------------------|-----------------|---------|---------|---------|----------|--------|
| <b>仅编码器 (无重排)</b>                  |                 |         |         |         |          |        |
| Qwen3-Emb-0.6B                     | –               | .587    | .533    | .560    | .638     | .595   |
| Qwen3-Emb-8B                       | –               | .653    | .627    | .640    | .698     | .659   |
| SR-Emb-0.6B                        | –               | .667    | .640    | .654    | .723     | .688   |
| <b>使用 nd 输入的重排器 (无主体)</b>          |                 |         |         |         |          |        |
| Qwen3-Emb-8B                       | Qwen3-Rank-8B   | .293    | .187    | .240    | .392     | .530   |
| Qwen3-Emb-0.6B                     | Qwen3-Rank-0.6B | .360    | .173    | .267    | .392     | .524   |
| Qwen3-Emb-8B                       | Qwen3-Rank-0.6B | .293    | .133    | .213    | .385     | .603   |
| Qwen3-Emb-8B                       | GPT-4o-mini     | .213    | .173    | .193    | –        | –      |
| Qwen3-Emb-0.6B                     | GPT-4o-mini     | .253    | .160    | .207    | –        | –      |
| Qwen3-Emb-8B                       | GPT-5.4-mini    | .347    | .267    | .307    | –        | –      |
| Qwen3-Emb-0.6B                     | GPT-5.4-mini    | .373    | .293    | .333    | –        | –      |
| <b>使用 full 输入的重排器——基础模型</b>        |                 |         |         |         |          |        |
| Qwen3-Emb-8B                       | GPT-4o-mini     | .667    | .627    | .647    | –        | –      |
| Qwen3-Emb-0.6B                     | GPT-4o-mini     | .560    | .547    | .554    | –        | –      |
| Qwen3-Emb-8B                       | GPT-5.4-mini    | .627    | .560    | .594    | –        | –      |
| Qwen3-Emb-0.6B                     | GPT-5.4-mini    | .573    | .547    | .560    | –        | –      |
| Qwen3-Emb-0.6B                     | Qwen3-Rank-0.6B | .653    | .627    | .640    | .684     | .604   |
| Qwen3-Emb-8B                       | Qwen3-Rank-0.6B | .613    | .547    | .580    | .672     | .694   |
| Qwen3-Emb-8B                       | Qwen3-Rank-8B   | .680    | .680    | .680    | .745     | .692   |
| <b>SR-Emb-0.6B + 重排器 (full 输入)</b> |                 |         |         |         |          |        |
| SR-Emb-0.6B                        | GPT-5.4-mini    | .667    | .653    | .660    | –        | –      |
| SR-Emb-0.6B                        | GPT-4o-mini     | .693    | .653    | .673    | –        | –      |
| SR-Emb-0.6B                        | Qwen3-Rank-0.6B | .720    | .693    | .707    | .769     | .724   |
| SR-Emb-0.6B                        | Qwen3-Rank-8B   | .720    | .707    | .714    | .776     | .727   |
| SR-Emb-0.6B                        | SR-Rank-0.6B    | .760    | .720    | .740    | .791     | .704   |
| <b>放大变体 (8B 组件)</b>                |                 |         |         |         |          |        |
| SR-Emb-0.6B                        | SR-Rank-8B      | .787    | .707    | .747    | .804     | .707   |
| SR-Emb-8B                          | SR-Rank-8B      | .787    | .733    | .760    | .808     | .719   |

## J 额外的流水线诊断

**重排器贡献分解。**表 22 显示，在 150 次 Easy+Hard 评估中，重排器修复了 19 个案例（12.7%）——这些案例中编码器未命中第 1 名但仍将正确技能检索进了 top-20 窗口——同时只使 6 个案例（4.0%）退化。因此净增益为 +8.7pp 的 Hit@1，从 65.3% 提升到 74.0%。其余 33 个未命中主要是召回失败，或需要多跳前提推理的案例。

## K 案例研究

我们呈现六个代表性案例，分析 SkillRouter 流水线的行为。正文聚焦于汇总结果；这里我们提供六个详细的成功与失败案例。

**案例 1：编码器优势 (video-tutorial-indexer)。**查询：从一个本地教程视频中提取章节时间戳。GT 技能：speech-to-text（基于 Whisper 的音频转录）。分析：基础编码器被表面关键词「video」误导，检索出视频编辑工具。Qwen3-Emb-0.6B 基础模型将 GT 排在第 25 位；Qwen3-Emb-8B 排在第 6 位。SR-Emb-0.6B 学会了间接映射「video + timestamps → speech-to-text」，并将 GT 检索到第 1 名。这表明微调捕获了单纯的模型规模所无法提供的推理捷径。

**案例 2：重排器挽救 (simpoc-code-reproduction)。**查询：复现一篇研究论文的损失函数并搭建开发环境。GT 技能：nlp-research-repo-package-installment（Python 环境搭建）。分析：所有编码器都错过了这一微妙的匹配（SR-Emb-0.6B：第 13 位；基础编码器：>50 位）。由于第 13 位在 top-20 窗口内，交叉编码器重排器识别出「搭建环境」与该技能的依赖安装指令之间的对齐，将 GT 提升到第 1 名。

**案例 3：编码器优势 (workflow-automation)。**查询：自动化一个带条件阶段执行的多步 CI/CD 流水线。GT 技能：github-actions-workflow（GitHub Actions YAML 生成）。分析：基础编码器检索出通用自动化技能（「task-scheduler」、「cron-manager」）。SR-Emb-0.6B 捕获了「CI/CD + 条件」→「GitHub Actions」的映射，将 GT 排在第 1 位，而 Qwen3-Emb-8B 排在第 8 位。该技能主体显式描述了条件工作流语法，而经微调的模型已学会将其与 CI/CD 查询关联。

**案例 4：重排器挽救 (data-format-conversion)。**查询：将一个遗留的 XML 配置转换为带模式校验的现代 TOML 格式。GT 技能：config-format-converter（多格式配置文件转换）。分析：编码器分别检索出聚焦 XML 和聚焦 TOML 的工具，但错过了统一转换器（SR-Emb-0.6B：第 11 位）。重排器通过对主体所支持格式列表（XML、YAML、JSON、TOML、INI）的交叉注意力，识别出正确的多格式技能并将其提升到第 1 名。

**案例 5：逐点损失退化 (api-documentation)。**查询：从 OpenAPI 规范生成带交互示例的 REST API 文档。GT 技能：openapi-doc-generator（Swagger/OpenAPI 文档工具）。分析：SR-Emb-0.6B 正确地将 GT 检索到第 1 名。然而，SR-Rank-0.6B (PW) 将其退化到第 18 位，而 SR-Rank-0.6B (LW) 保持第 1 名。逐点模型为所有 API 相关候选分配了相似的分值（约 0.52），实际上把顺序随机化了。该案例例证了为什么逐点评分在同质候选池中会失败。

**案例 6: 系统局限 (invoice-fraud-detection)。**查询: 分析发票图像以检测潜在的欺诈模式 (重复金额、可疑供应商名称)。GT 技能: pdf-table-extractor (从 PDF/图像中提取结构化数据)。分析: 所有方法都失败了。「发票欺诈检测」与「PDF 表格提取」之间的关联需要多跳推理: 欺诈分析 → 需要结构化数据 → 发票是 PDF → 表格提取。我们流水线中没有任何检索方法捕获到这一链条。顶端结果是欺诈检测分析工具和图像分类器, 它们都不提供前提性的数据提取步骤。该案例说明了我们当前评估中剩余未命中率的一个重要来源: 基于检索的流水线在需要多跳前提推理的任务上仍然困难。

**表 22:** 逐查询 Hit@1 分解: SR-Emb-0.6B 仅编码器 vs. 完整的 SR-Emb-0.6B × SR-Rank-0.6B 流水线。

| 类别        | Easy n | Easy % | Hard n | Hard % | All n | All % |
|-----------|--------|--------|--------|--------|-------|-------|
| 两者均正确     | 47     | 62.7   | 45     | 60.0   | 92    | 61.3  |
| 重排器修复     | 10     | 13.3   | 9      | 12.0   | 19    | 12.7  |
| 重排器退化     | 3      | 4.0    | 3      | 4.0    | 6     | 4.0   |
| 两者均未命中    | 15     | 20.0   | 18     | 24.0   | 33    | 22.0  |
| 编码器 Hit@1 | 50     | 66.7   | 48     | 64.0   | 98    | 65.3  |
| 流水线 Hit@1 | 57     | 76.0   | 54     | 72.0   | 111   | 74.0  |

## L 下游评估细节

我们报告支撑正文平均表格的完整直接执行结果。所有运行在 75 任务核心集上使用相同的执行框架, 超时设为 1200s。本附录报告的是正文中所描述的、使用自然池的相同互补式端到端研究, 而非 Easy/Hard 检索基准的分层匹配延续。被检索技能来自自然池, 即不含 Hard 分层干扰项的非合成基准池。一旦暴露的技能包被固定, 下游执行协议在其余方面遵循 SkillsBench (Li et al., 2026b), 包括任务设置和成功校验。四个被评估的智能体是 Kimi-K2.5 (Kimi Team, 2026)、glm-5 (Z.AI, 2026)、Claude Sonnet 4.6 和 Claude Opus 4.6 (Anthropic, 2026)。表 23 报告了支撑表 6 中平均正文结果的各智能体成功率。

### L.1 代表性下游案例

这些案例说明了路由质量如何影响端到端的智能体执行。它们是从 3 次试验评估中抽取的机制说明, 而非统计证明。

**案例 A(正面): 语义干扰项规避 (software-dependency-audit)。**查询: 审计项目依赖的已知漏洞, 提取 CVSS 分数, 并生成一份 CSV 报告。黄金技能: cvss-score-extraction、trivy-offline-vulnerability-scanning、vulnerability-csv-reporting。检索: 基线 top-1 检索出 dependency-security (一个关于通用依赖安全的社区贡献技能), 而 SkillRouter top-1 检索出 trivy-offline-vulnerability-scanning (一个提供特定离线扫描工作流的黄金技能)。结果: 基线 top-1 在所有四个智能体上得分 0/12; SkillRouter

top-1 得分 12/12。当正确技能被路由时，每个智能体都从完全失败转变为完全成功。这是数据集中最具戏剧性的案例，说明了当选错技能时，语义相似但功能不同的技能如何能完全阻断下游执行。

**案例 B（正面）：表面匹配陷阱 (video-tutorial-indexer)。**查询：通过提取并加时间戳其口语内容来为一个视频教程建立索引。黄金技能：speech-to-text。检索：基线 top-1 检索出 video-explorer (匹配表面关键词「video」)，而 SkillRouter 检索出 speech-to-text (匹配所需功能：转录)。结果：基线 top-1 得分 0/12；SkillRouter top-1 得分 9/12 (glm-5 3/3, Kimi-K2.5 2/3, Opus 3/3, Sonnet 1/3)。基线检索器匹配的是任务的表面主题（视频），而非其功能需求（音频转录），这是经训练的路由器所规避的一种失败模式。

**案例 C（负面）：专业化多技能领域 (hvac-control)。**查询：设计并调优一个带安全互锁的模型预测 HVAC 控制器。黄金技能：excitation-signal-design、first-order-model-fitting、imc-tuning-rules、safety-interlocks、scipy-curve-fit。检索：基线 top-1 检索出 first-order-model-fitting (一个黄金技能)；SkillRouter top-1 检索出 simulation-metrics (一个相关但非黄金的控制工程技能)。在 top-10 设置下，基线恢复了全部 5 个黄金技能，而 SkillRouter 只恢复了 1 个。结果：基线 top-1 得分 8/12；SkillRouter top-1 得分 4/12。该案例说明了紧凑路由器的不足之处：在需要跨多个子学科进行精确词汇匹配的高度专业化多技能工程领域中，16B 基线更深的领域表示产生了实质上更好的检索。



**表 23:** 按模型与技能条件划分的直接端到端智能体成功率 (%)。每个任务在每种条件下评估三次。检索技能使用自然池，即不含 Hard 分层干扰项的非合成基准池。

| 模型                       | 技能条件 | 路由器/来源             | Top-K | 单技能   | 多技能   | 总体    |
|--------------------------|------|--------------------|-------|-------|-------|-------|
| <b>Kimi-K2.5</b>         | 无技能  | 无                  | –     | 12.50 | 9.15  | 10.22 |
|                          | 黄金技能 | Oracle 黄金标准        | GT    | 20.83 | 25.49 | 24.00 |
|                          | 检索技能 | Qwen3-Pipeline-16B | 1     | 20.83 | 11.76 | 14.67 |
|                          | 检索技能 | SkillRouter-1.2B   | 1     | 20.83 | 16.34 | 17.78 |
|                          | 检索技能 | Qwen3-Pipeline-16B | 10    | 16.67 | 21.57 | 20.00 |
|                          | 检索技能 | SkillRouter-1.2B   | 10    | 19.44 | 17.65 | 18.22 |
| <b>glm-5</b>             | 无技能  | 无                  | –     | 11.11 | 12.42 | 12.00 |
|                          | 黄金技能 | Oracle 黄金标准        | GT    | 37.50 | 23.53 | 28.00 |
|                          | 检索技能 | Qwen3-Pipeline-16B | 1     | 36.11 | 19.61 | 24.89 |
|                          | 检索技能 | SkillRouter-1.2B   | 1     | 36.11 | 16.99 | 23.11 |
|                          | 检索技能 | Qwen3-Pipeline-16B | 10    | 19.44 | 21.57 | 20.89 |
|                          | 检索技能 | SkillRouter-1.2B   | 10    | 30.56 | 22.22 | 24.89 |
| <b>Claude Sonnet 4.6</b> | 无技能  | 无                  | –     | 13.89 | 22.22 | 19.56 |
|                          | 黄金技能 | Oracle 黄金标准        | GT    | 40.28 | 43.79 | 42.67 |
|                          | 检索技能 | Qwen3-Pipeline-16B | 1     | 31.94 | 34.64 | 33.78 |
|                          | 检索技能 | SkillRouter-1.2B   | 1     | 36.11 | 37.25 | 36.89 |
|                          | 检索技能 | Qwen3-Pipeline-16B | 10    | 33.33 | 38.56 | 36.89 |
|                          | 检索技能 | SkillRouter-1.2B   | 10    | 36.11 | 42.48 | 40.44 |
| <b>Claude Opus 4.6</b>   | 无技能  | 无                  | –     | 12.50 | 20.26 | 17.78 |
|                          | 黄金技能 | Oracle 黄金标准        | GT    | 25.00 | 41.18 | 36.00 |
|                          | 检索技能 | Qwen3-Pipeline-16B | 1     | 18.06 | 35.29 | 29.78 |
|                          | 检索技能 | SkillRouter-1.2B   | 1     | 26.39 | 35.29 | 32.44 |
|                          | 检索技能 | Qwen3-Pipeline-16B | 10    | 12.50 | 29.41 | 24.00 |
|                          | 检索技能 | SkillRouter-1.2B   | 10    | 18.06 | 32.03 | 27.56 |

**表 24:** 一个尺寸匹配的 1.2B 基础流水线、最强的 16B 基础流水线，以及主 1.2B SkillRouter 流水线的分层鲁棒性总结。Easy 和 Hard 报告主基准上分层特定的 Hit@1；Single 和 Multi 报告单技能和多技能查询子集上的 Hit@1；Multi FC@10 报告多技能查询在前 10 名的严格完整覆盖。

| 流水线                                     | Easy | Hard | Single | Multi | Multi FC@10 |
|-----------------------------------------|------|------|--------|-------|-------------|
| Qwen3-Emb-0.6B $\times$ Qwen3-Rank-0.6B | .653 | .627 | .625   | .647  | .324        |
| Qwen3-Emb-8B $\times$ Qwen3-Rank-8B     | .680 | .680 | .667   | .686  | .382        |
| SR-Emb-0.6B $\times$ SR-Rank-0.6B       | .760 | .720 | .729   | .745  | .353        |

**表 25:** 完整的重排器损失消融。编码器 = SR-Emb-0.6B，top-20 候选。LW = 列表式交叉熵；PW = 逐点二元交叉熵；base = 未调优。

| 重排器               | 损失   | Easy Hit@1 | Easy MRR@10 | Hard Hit@1 | Hard MRR@10 | Avg Hit@1 | Avg MRR@10 |
|-------------------|------|------------|-------------|------------|-------------|-----------|------------|
| SR-Rank-0.6B (LW) | LW   | .760       | .809        | .720       | .773        | .740      | .791       |
| Qwen3-Rank-0.6B   | base | .720       | .780        | .693       | .758        | .707      | .769       |
| Qwen3-Rank-8B     | base | .720       | .781        | .707       | .771        | .714      | .776       |
| SR-Rank-0.6B (PW) | PW   | .453       | .592        | .413       | .564        | .433      | .578       |
| 仅编码器（无重排）         |      | .667       | .735        | .640       | .710        | .654      | .723       |

## M 补充基准：SkillBench-Supp

本附录提供第 5.4 节所引用补充基准的完整构建细节、查询生成提示词、质量保证措施和完整结果。

### M.1 GT 技能来源

这 100 个 GT 技能取自三个独立来源以确保广度：

- **OpenClaw 官方仓库 (51 个技能):** 来自官方 OpenClaw 技能仓库 (OpenClaw, 2026a) 的内置技能，覆盖通信、生产力、IoT、媒体、安全等领域。每个技能都是一个带有 name、description 和 body 字段的标准 SKILL.md 文档。
- **AgentSkillOS 基准 (19 个技能):** 来自 AgentSkillOS 评估集 (Li et al., 2026a) 的 GT 技能，覆盖文档处理、可视化、前端设计等类似领域。纳入它们是为了检验与现有技能基准的一致性。
- **池选技能 (30 个技能):** 从 78K 基础池中跨 25 个应用领域（游戏、旅行、安全、数据科学等）分层采样，仅选取名称唯一、主体长度在 1,000–15,000 字符之间的技能，以确保长尾领域中的质量与多样性。

## M.2 池构建

候选池源自与主基准所用相同的约 78K 基础池。为防止假阴性——即某个池条目与某个 GT 技能功能上相同却被判为不正确——我们应用两个去重步骤:

1. **内容去重:** 移除与任何 GT 技能内容相同或近似相同的条目。
2. **功能重叠移除:** 对每个 GT 技能, 我们通过 BM25 (以 name + description + body 作为文档) 检索 top-5 最相似的池条目, 然后对每一对应用基于 LLM (Claude Sonnet) 的等价性判定。被判为功能等价的条目被移除。这一过程在 49 个 GT 技能上识别并移除了 114 个等价对 (其中 github、docx、xlsx 和 pptx 各有 5 个等价条目——这是观察到的最大值)。

最终池包含 77,537 个技能。

## M.3 查询生成

对每个 GT 技能, 使用 Claude Sonnet (claude-sonnet-4-6) 在两个难度级别上、以 temperature = 0.8 生成 2-3 条评价查询:

**描述性查询 (共 168 条)。**这些查询描述一个具体的使用场景而不点名该技能。两种子风格确保了风格多样性:

场景子风格 (82 条查询, 80-250 词): 带有具体上下文的详细任务描述。

Given this skill specification, write a realistic user query describing a task that requires this skill's capabilities. Skill name: {name}. Description: {description}. Skill content preview: {body}. Requirements: (1) Describe a specific task or problem the user wants to solve. (2) Include enough context for the skill to be clearly the right one. (3) Write naturally — as a user would describe their need to an AI assistant. (4) Do NOT follow a rigid structure (no mandatory file paths, no mandatory numbered lists). (5) Total length: 80-250 words. (6) Do NOT mention the skill name "{name}" or reference this specification document. (7) Do NOT use any unique identifiers or CLI command names from the skill content. Output ONLY the user query. No preamble, no explanation.

开发者子风格 (86 条查询, 40-120 词): 简洁、自然的请求。

Given this skill specification, write a concise user query from someone who needs this capability but doesn't know the specific skill exists. Skill name: {name}. Description: {description}. Skill content preview: {body}. Requirements: (1) Write as a user naturally asking for help — could be casual or formal. (2) Describe what they want to achieve, not what skill to use. (3) Be specific enough that this skill is clearly the best match. (4) Total length: 40-120 words. (5) Do NOT mention the skill name "{name}" or any CLI commands from the skill. (6) Do NOT reference this specification document. Output ONLY the user query.

**间接查询 (共 88 条)。**这些查询描述一个需要进行能力推断才能与该技能关联起来的高层需求 (50–180 词)。

Given this skill specification, write a user query that describes a high-level need which this skill would address, without revealing the skill itself. Skill name: {name}. Description: {description}. Skill content preview: {body}. Requirements: (1) Describe a real-world problem or goal, not a skill request. (2) The user should not know this specific skill exists — they’re describing their situation. (3) Do NOT mention the skill name “{name}” or any specific CLI commands from the skill. (4) Avoid using keywords that directly appear in the skill’s name or description. (5) The connection between the query and this skill should require understanding the skill’s capabilities. (6) Total length: 50–180 words. (7) Be specific enough that this skill is the clear best answer (not too vague). Output ONLY the user query.

**自动化质量控制。**每条生成的查询都经过三项检查: (1) 技能名称泄露检测, (2) CLI 命令名称泄露检测, (3) 长度合规性。不合规的查询会被自动重新生成 (最多 3 次尝试)。最终数据集包含 256 条查询, 覆盖 100 个 GT 技能中的 93 个 (查询长度: 最小 = 43, 最大 = 226, 均值 = 117 词)。

## M.4 质量保证

若干设计选择确保 SkillBench-Supp 提供了一个可信、无偏的评估:

1. **不同的生成方法论:** 评估查询由 Claude Sonnet (claude-sonnet-4-6) 生成, 而 SkillRouter 训练查询由 GPT-4o-mini 以不同的提示词设计和技能采样策略生成。这一分离防止了评估查询与训练数据在模型、提示词模板和来源分布方面的任何重叠。
2. **自动化泄露过滤:** 每条查询都检查三类泄露: (a) 技能名称子串匹配, (b) 针对技能主体的 CLI 命令名称匹配, (c) 与目标难度级别的长度合规性。不合规的查询最多重新生成 3 次后才被丢弃。
3. **功能重叠移除:** 如第 M.2 节所述, 通过 BM25 + 基于 LLM 的等价性判定识别并移除了 114 个功能等价的池条目, 防止评估指标中的假阴性污染。
4. **GT 技能隔离:** 30 个池选 GT 技能被显式地排除在训练 GT 技能集之外, 因此该补充基准不会重用来自基础池的训练正例技能。
5. **多来源 GT 技能:** 这 100 个 GT 技能跨越三个独立来源 (官方仓库、现有基准和池选) 以及 25 个以上的应用领域, 降低了系统性偏向任何特定技能类型或领域的风险。

## M.5 完整结果

表 26 报告了在 77K 池、256 条单标签查询上的完整纯编码器及编码器 × 重排器结果。

**表 26:** SkillBench-Supp: 完整的纯编码器及编码器  $\times$  重排器结果 (77K 池, 256 条单标签查询)。R@50 反映了在编码器 top-50 候选上进行下游重排的理论上限, 因此仅对纯编码器行有定义; 流水线行标记为「-」。

| 系统                                      | H@1  | H@3  | H@5  | H@10 | H@20 | R@50 | MRR@10 |
|-----------------------------------------|------|------|------|------|------|------|--------|
| <b>纯编码器检索</b>                           |      |      |      |      |      |      |        |
| BM25                                    | .066 | .094 | .125 | .141 | .199 | .250 | .088   |
| Qwen3-Emb-0.6B                          | .461 | .578 | .633 | .707 | .773 | .840 | .537   |
| Qwen3-Emb-8B                            | .504 | .609 | .676 | .742 | .809 | .867 | .575   |
| SR-Emb-0.6B                             | .488 | .617 | .668 | .723 | .797 | .848 | .566   |
| SR-Emb-8B                               | .605 | .734 | .762 | .824 | .871 | .918 | .675   |
| <b>「检索-重排」流水线</b>                       |      |      |      |      |      |      |        |
| Qwen3-Emb-0.6B $\times$ Qwen3-Rank-0.6B | .516 | .664 | .707 | .781 | .812 | -    | .603   |
| Qwen3-Emb-8B $\times$ Qwen3-Rank-8B     | .637 | .762 | .801 | .852 | .863 | -    | .707   |
| SR-Emb-0.6B $\times$ Qwen3-Rank-0.6B    | .574 | .707 | .762 | .805 | .828 | -    | .652   |
| SR-Emb-0.6B $\times$ SR-Rank-0.6B       | .641 | .746 | .781 | .809 | .840 | -    | .699   |
| SR-Emb-8B $\times$ Qwen3-Rank-8B        | .699 | .816 | .855 | .895 | .914 | -    | .764   |
| SR-Emb-8B $\times$ SR-Rank-8B           | .730 | .828 | .859 | .895 | .910 | -    | .784   |

## M.6 难度细分

表 27 和表 28 报告了纯编码器和流水线系统在描述性与间接查询上的难度细分。间接查询在所有系统上都一致地更难, 其中 BM25 上的难度差距最大 (H@1 从 .083 降至 .034), 证实了间接查询需要更深的语义理解。

表 27: SkillBench-Supp: 描述性查询上的结果 (168 条; 77K 池)。R@50 仅对纯编码器检索有定义, 故流水线行标记为「-」。

| 系统                               | H@1  | H@3  | H@5  | H@10 | H@20 | R@50 | MRR@10 |
|----------------------------------|------|------|------|------|------|------|--------|
| <b>纯编码器检索</b>                    |      |      |      |      |      |      |        |
| BM25                             | .083 | .113 | .143 | .161 | .226 | .286 | .105   |
| Qwen3-Emb-0.6B                   | .500 | .613 | .673 | .750 | .815 | .875 | .577   |
| Qwen3-Emb-8B                     | .560 | .637 | .708 | .756 | .827 | .893 | .618   |
| SR-Emb-0.6B                      | .560 | .679 | .720 | .762 | .827 | .881 | .629   |
| SR-Emb-8B                        | .673 | .780 | .798 | .851 | .887 | .929 | .729   |
| <b>「检索-重排」流水线</b>                |      |      |      |      |      |      |        |
| Qwen3-Emb-0.6B × Qwen3-Rank-0.6B | .524 | .690 | .738 | .798 | .833 | -    | .620   |
| Qwen3-Emb-8B × Qwen3-Rank-8B     | .661 | .804 | .845 | .875 | .887 | -    | .738   |
| SR-Emb-0.6B × Qwen3-Rank-0.6B    | .601 | .744 | .780 | .827 | .857 | -    | .682   |
| SR-Emb-0.6B × SR-Rank-0.6B       | .679 | .786 | .821 | .839 | .881 | -    | .736   |
| SR-Emb-8B × Qwen3-Rank-8B        | .726 | .845 | .875 | .911 | .923 | -    | .789   |
| SR-Emb-8B × SR-Rank-8B           | .786 | .851 | .881 | .923 | .929 | -    | .828   |

表 28: SkillBench-Supp: 间接查询上的结果 (88 条; 77K 池)。R@50 仅对纯编码器检索有定义, 故流水线行标记为「-」。

| 系统                               | H@1  | H@3  | H@5  | H@10 | H@20 | R@50 | MRR@10 |
|----------------------------------|------|------|------|------|------|------|--------|
| <b>纯编码器检索</b>                    |      |      |      |      |      |      |        |
| BM25                             | .034 | .057 | .091 | .102 | .148 | .182 | .055   |
| Qwen3-Emb-0.6B                   | .386 | .511 | .557 | .625 | .693 | .773 | .460   |
| Qwen3-Emb-8B                     | .398 | .557 | .614 | .716 | .773 | .818 | .494   |
| SR-Emb-0.6B                      | .352 | .500 | .568 | .648 | .739 | .784 | .446   |
| SR-Emb-8B                        | .477 | .648 | .693 | .773 | .841 | .898 | .573   |
| <b>「检索-重排」流水线</b>                |      |      |      |      |      |      |        |
| Qwen3-Emb-0.6B × Qwen3-Rank-0.6B | .500 | .614 | .648 | .750 | .773 | -    | .570   |
| Qwen3-Emb-8B × Qwen3-Rank-8B     | .591 | .682 | .716 | .807 | .818 | -    | .647   |
| SR-Emb-0.6B × Qwen3-Rank-0.6B    | .523 | .636 | .727 | .761 | .773 | -    | .595   |
| SR-Emb-0.6B × SR-Rank-0.6B       | .568 | .670 | .705 | .750 | .761 | -    | .627   |
| SR-Emb-8B × Qwen3-Rank-8B        | .648 | .761 | .818 | .864 | .898 | -    | .717   |
| SR-Emb-8B × SR-Rank-8B           | .625 | .784 | .818 | .841 | .875 | -    | .701   |